GhostStream: Real-Time Video Inpainting via Memory-Guided Conditional Synthesis for Dynamic Cameras

Botsian Liu[#1], Devon Ragosin[#2], Junyi Li[#3]

[#]*Computer Science, University of Alberta*

[1]bqian1@ualberta.ca

[2]ragosin@ualberta.ca

[3]junyi19@ualberta.ca

Abstract

Real-time background replacement in video typically relies on static cameras or controlled environments like green screens. When a camera moves, revealing previously occluded background areas, traditional systems fail to maintain temporal consistency without severe computational overhead. This paper presents a dynamic, real-time background subtraction and inpainting pipeline designed for moving cameras. Our system combines MAGSAC++ homography estimation with a frame-to-frame memory buffer to track camera motion and synthesize missing background data. By dynamically splitting the occlusion mask into a "Temporal Zone" (known memory) and a "Spatial Zone" (unknown areas requiring hallucination), we drastically reduce the computational load. We evaluate the spatial zone synthesis using both classical patch-based methods (PatchMatch) and modern deep-learning image inpainting (LaMa). Our results demonstrate that conditional routing of these synthesis engines allows the pipeline to bypass heavy network inferences during static or known-view periods, maintaining high real-time frame rates on edge devices like Apple Silicon while dynamically adapting to environmental changes.

In addition, an offline video inpainting approach is explored for more challenging cases. This approach is based on flow-guided video inpainting methods such as E2FGVI, which rely on optical flow to propagate known pixel information across frames, followed by a hallucination stage to reconstruct regions that cannot be filled through propagation. By using both forward and backward optical flow, these methods are able to maintain temporal consistency and reuse information from different frames, improving reconstruction quality in dynamic scenes. Compared with the real-time pipeline, this approach provides stronger reconstruction performance when real-time constraints are relaxed. Based on this framework, potential improvements are considered, including more accurate optical flow estimation and more efficient attention mechanisms in the hallucination stage, which could further improve both visual quality and computational efficiency.

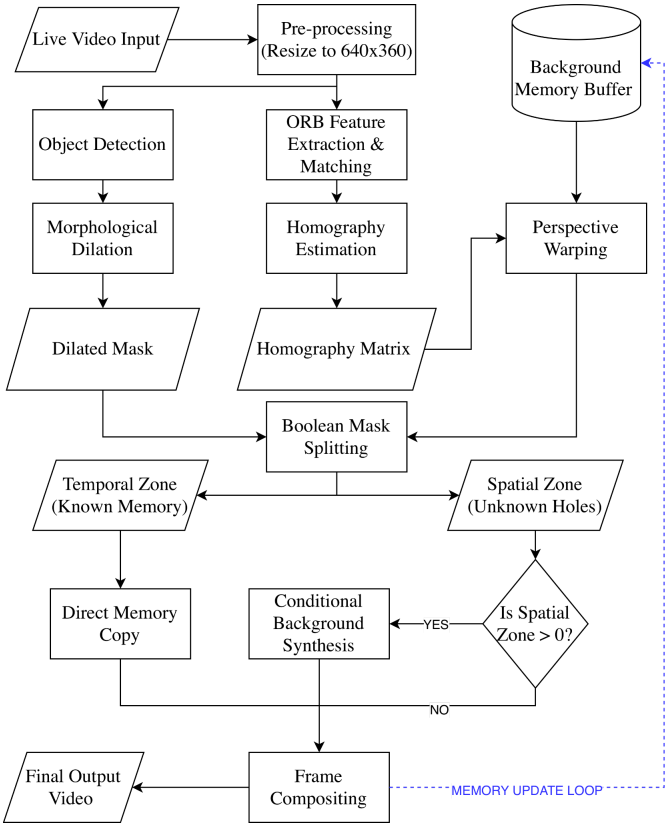


Figure 1. System architecture and data flow of the proposed real-time background subtraction and conditional synthesis pipeline.

Demo videos and project codes are available at https://github.com/BotsianLio/GhostStream.git

1. Introduction

Background subtraction and replacement are foundational tasks in computer vision, widely used in video conferencing, augmented reality, and film production. Historically, flawless background manipulation required either a completely static camera or a physical chroma-key (green screen) backdrop. If a user moves the camera in a natural, dynamic setting, the background shifts, revealing "holes"—areas of the environment that were previously hidden behind the foreground subject.

To create a seamless illusion, a system must not only track the movement of the background but also hallucinate the missing pixels in real-time. Recent advancements in deep learning have introduced highly capable video inpainting models, such as E2FGVI[1]. These methods typically follow a three-stage pipeline consisting of optical flow estimation, feature propagation across neighboring frames, and a final hallucination stage to reconstruct regions that cannot be filled through propagation. Within this framework, additional components such as feature encoding and transformer-based modules are used to improve efficiency and reconstruction quality. By leveraging both forward and backward optical flow, such approaches are able to reuse information from different temporal contexts and maintain strong temporal consistency in dynamic scenes. An illustration of this pipeline is shown in Figure 2 [1].

However, these models often rely on massive temporal transformers or 3D convolutions that demand datacenter-grade GPUs, rendering them unusable for live, real-time applications on consumer hardware.

To bridge this gap, we propose a hybrid pipeline that achieves dynamic background replacement without the continuous computational penalty of video-based neural networks. Our system utilizes a dynamic memory canvas that mathematically aligns past frames to the current camera view using MAGSAC++[2] feature matching. By applying Boolean logic to isolate only the newly revealed unknown pixels, the system conditionally triggers synthesis models such as classical PatchMatch[3] or the deep-learning-based LaMa[4] architecture only when absolutely necessary. Therefore, efficient reconstruction can be achieved while maintaining real-time performance. In addition to the implemented real-time system, offline video inpainting methods are examined to better understand their advantages and disadvantages. Flow-guided approaches such as E2FGVI[1] are analyzed in terms of their ability to propagate information across frames and maintain temporal consistency. Based on this analysis, we propose two key improvements. By implementing these modifications, we demonstrate strong empirical results on the DAVIS[5] and YouTube-VOS 2019[6] test sets, validating that these architectural changes significantly boost reconstruction metrics.

This paper details the mathematical architecture, evaluates the performance trade-offs on edge devices, and discusses the inherent challenges of memory contamination and temporal inconsistency. At the same time, combining a practical real-time pipeline with an analysis of offline methods provides insight into

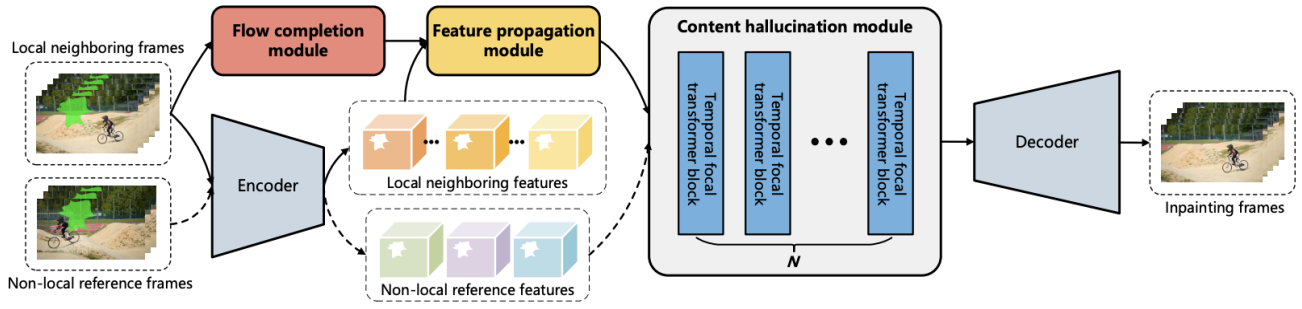


Figure 2. Overview of the proposed End-to-End framework for Flow-Guided Video Inpainting (E^2FGVI). It consists of 1) a frame-level content encoder, 2) a flow completion module, 3) a feature propagation module, 4) a content hallucination module which is composed of multiple temporal focal transformer blocks, and 5) a frame-level decoder.

the trade-offs between efficiency and reconstruction quality, as well as possible directions for improving existing approaches.

2. Related Work

2.1 Classical Background Modeling and Synthesis

Traditional background subtraction relies on statistical models of the environment over time. However, these methods fail instantly under camera motion. To handle missing data (inpainting), classical algorithms like PatchMatch have been widely utilized. PatchMatch relies on Nearest Neighbor Fields (NNF)[3] to efficiently search and copy-paste texture patches from known regions of an image into the missing holes. While computationally lightweight and effective for repetitive textures, classical methods lack semantic understanding and struggle to synthesize complex structural elements spanning large gaps.

2.2 Deep Learning Video Inpainting

The advent of deep learning has revolutionized video inpainting. State-of-the-art models such as STTN[5](Spatial-Temporal Transformer Networks) and E²FGVI (Efficient Explicit Flow-Guided Video Inpainting) achieve remarkable temporal consistency by processing multiple frames simultaneously and utilizing optical flow to warp features across time. While models like E²FGVI[1] report highly efficient inference times (e.g., 0.16s per frame) on high-end desktop GPUs, our preliminary testing revealed that these complex architectures are largely incompatible or prohibitively slow when deployed on consumer ARM-based edge devices (such as Apple Silicon), dropping throughput to unusable frame rates for live applications.

2.2.1 Flow-Guided Video Inpainting and E2FGVI

Flow-guided video inpainting approaches, such as E2FGVI [1] (End-to-End Flow Guided Video Inpainting), are among the current state-of-the-art methods. These approaches rely on optical flow, typically both forward and backward, to propagate known pixel values from previous or future frames to fill missing regions in the current frame. For regions that cannot be filled through propagation, a separate inpainting stage, often referred to as hallucination, is used to generate the remaining content.

In general, flow-guided methods follow the 3 stage pipeline consisting of optical flow estimation, pixel propagation, and a final inpainting stage. In the first stage, optical flow is computed between neighboring frames, typically in both forward and backward directions, to capture motion and establish temporal correspondences. For efficiency, this step is often performed on downsampled inputs and limited to adjacent frames. In the second stage, the estimated flow is used to guide feature propagation across frames. Instead of directly copying raw pixels, propagation is typically performed in feature space, allowing the model to better handle inaccuracies in motion estimation. By leveraging bidirectional flow, the model can reuse valid information from other frames and wrap it into the current frame, improving structural consistency. In the final stage, a hallucination module is applied to reconstruct regions that cannot be filled through propagation alone. This stage is commonly implemented using temporal transformer architectures (e.g. temporal focal transformers) that utilize temporal context to generate plausible content, followed by a decoder to restore the output to the original resolution.

A key advantage of this approach is the efficiency of its optical flow estimation, which is performed in a single pass and allows faster processing compared to earlier methods, albeit at the cost of reduced accuracy. To mitigate this limitation, optical flow estimation and feature propagation are jointly optimized, allowing the propagation stage to compensate for inaccuracies in the estimated motion and still produce consistent results. Additionally, operating in an encoded or downsampled feature space reduces computational cost and improves overall efficiency.

Other flow-guided video inpainting methods include FGVC [6] and DFVI [7], which follow similar multi-stage pipelines based on optical flow

and feature propagation. In these approaches, each stage is typically implemented and trained as a separate module. In contrast, E2FGVI[1] integrates all stages into a single end-to-end network that is trained jointly.

2.2.2 Attention-Based Methods (STTN[5])

Attention-based video inpainting methods have been proposed to better capture long-range correspondences across frames. A typical example is STTN[5] (Spatial-Temporal Transformer Networks), which introduces a joint spatial-temporal transformer framework for video inpainting. Unlike approaches that process videos frame by frame, STTN[5] formulates video inpainting as a "multi-to-multi" problem and simultaneously fills missing regions in all input frames.

Specifically, STTN[5] takes both neighboring frames and distant frames as input conditions, allowing the model to preserve temporal consistency in local regions while maintaining coherence with the overall video context. The main component of STTN[5] is a spatial-temporal transformer that searches for relevant content across frames using a multi-scale patch-based attention mechanism. In this design, patches of different spatial scales are extracted from all frames, and attention is computed across these patches to identify and aggregate relevant information for reconstructing missing regions.

To further improve reconstruction quality, multiple spatial-temporal transformer layers are stacked, enabling progressive refinement of the inpainted regions. In addition, the model is optimized using both pixel-wise reconstruction losses and a spatial-temporal adversarial loss to ensure quality and temporal consistency.

However the model may still produce blurry results in cases involving large missing regions and continuous rapid motion, showing the limitation in modeling short-term temporal dynamics under complex motion conditions.

2.3 Image-Based Neural Inpainting

To maintain real-time performance on edge devices, we look to single-image inpainting networks. Models like LaMa[4] (Resolution-robust Large Mask Inpainting) utilize Fast Fourier Convolutions (FFCs) to capture a global receptive field, allowing the network to understand the entire image context rather than relying solely on local textures. By implementing LaMa[4] inside a conditional spatial-zone trigger, our proposed method leverages the semantic power of modern deep learning while bypassing the massive computational bottleneck of multi-frame video models.

3. Methodology

The proposed pipeline is designed to perform real-time background replacement under dynamic camera motion while minimizing computational overhead. The architecture is divided into four distinct stages: pre-processing the occlusion mask, motion compensation via robust homography estimation, dynamic zone segmentation using Boolean logic, and conditional background synthesis.

3.1 Pre-processing and Morphological Dilation

For any given timestamp t, the system receives an input RGB frame I_t. A pre-trained object detection model is applied to identify the foreground subject, generating an initial binary occlusion mask M_t.

A critical challenge in live video processing is motion blur, which often extends beyond the tight bounding contours generated by standard object detectors. If these blurred transient pixels are not masked, they will contaminate the background memory buffer, resulting in persistent silhouette artifacts. To mitigate this, we apply a morphological dilation to the initial mask. Using a structural element (kernel) K of size 15×15, the dilated mask D_t is computed as:

$$D_t(x, y) = \max_{(i,j) \in K} M_t(x + i, y + j) \qquad (1)$$

This dilation ensures that peripheral motion blur is securely encapsulated within the occlusion region.

3.2 Motion Compensation

To maintain temporal consistency when the camera moves, the system utilizes a frame-to-frame memory buffer, denoted as B_{t-1}. Before this memory can be applied to the current frame, it must be geometrically aligned. We extract ORB[8] (Oriented FAST and Rotated BRIEF) feature points from both I_{t-1} and I_t to compute the 3×3 planar homography matrix H:

$$\begin{bmatrix} x'w \\ y'w \\ w \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (2)$$

Because the moving foreground subject generates severe outlier feature matches, a robust estimator is required. Standard RANSAC[9] relies on a hard, binary inlier threshold, which often rejects slightly noisy valid background points, causing noticeable jitter in the resulting alignment. Instead, we implement MAGSAC++[2] (Marginalizing Sample Consensus). Rather than utilizing a fixed threshold, MAGSAC++[2] assigns a continuous weight to each data point by marginalizing over a range of possible noise standard deviations (σ):

$$W(r) = \int \text{Loss}(r|\sigma)p(\sigma)d\sigma \quad (3)$$

where r represents the residual error. This probabilistic weighting significantly smooths the frame-to-frame tracking, producing a highly stable homography matrix.

3.3 Dynamic Zone Segmentation

Using the computed homography H, the previous background memory is warped into the current camera perspective to create the aligned memory canvas W_t:

$$W_t(x, y) = B_{t-1}\left(H^{-1} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}\right) \quad (4)$$

Due to camera panning, certain regions of W_t will fall outside the original field of view, returning null pixels. We define a valid memory mask V_t representing all coordinates where $W_t(x, y) \neq 0$. To optimize computational efficiency, the dilated occlusion mask D_t is split into two distinct operational zones using Boolean logic:

3.3.1 Temporal Zone (Z_{temp}):

The region where the background is currently occluded by the subject, but a valid historical memory exists.

$$Z_{temp} = D_t \wedge V_t \quad (5)$$

3.3.2 Spatial Zone (Z_{spatial}):

The region where the background is occluded, but no historical memory exists (i.e., newly revealed spatial areas due to camera motion).

$$Z_{\text{spatial}} = D_t \wedge \neg V_t \quad (6)$$

3.4 Conditional Background Synthesis & Memory Update

The core efficiency of this pipeline stems from its conditional routing. Deep learning inference is entirely bypassed unless $\sum Z_{\text{spatial}} > 0$. When a spatial unknown is detected, the system triggers the synthesis module to hallucinate the missing pixels. We evaluate two differing paradigms for this synthesis: Classical Approach (PatchMatch[3]) and Deep Learning Approach (LaMa[4]). Finally, the synthesized data is composited into the final output frame O_t, and the background memory B_t is updated for the next iteration. The output compositing logic is defined as:

$$O_t(x, y) = \begin{cases} W_t(x, y), & \text{if } Z_{\text{temp}}(x, y) = \\ \text{Synthesis}(x, y), & \text{if } Z_{\text{spatial}}(x, y) \\ I_t(x, y), & \text{if } D_t(x, y) = 0 \end{cases} \quad (7)$$

Concurrently, the memory buffer strictly updates using the live, unoccluded camera feed to learn new environments, while preserving the warped memory beneath the subject:

$$B_t(x,y) = \begin{cases} I_t(x,y), & \text{if } D_t(x,y) = 0 \\ W_t(x,y), & \text{if } D_t(x,y) = 1 \end{cases} \quad (8)$$

4. Experiments & Results

To validate the effectiveness of the proposed dynamic background replacement pipeline, we conducted evaluations focusing on both visual fidelity and computational efficiency.

4.1 Experimental Setup

The pipeline was evaluated on a custom recorded video sequence consisting of 179 frames, featuring dynamic camera panning and active foreground subject motion. The system was deployed on an Apple Silicon environment (MPS/CPU) to benchmark performance on non-CUDA edge hardware.

4.2 Evaluation Metrics

Peak Signal-to-Noise Ratio (PSNR): Measures the absolute pixel-level error between the reconstructed background and the clean ground truth. Higher is better.

Structural Similarity Index (SSIM): Evaluates the perceived structural coherence and texture degradation. Values range from 0 to 1, where higher indicates better structural preservation.

4.3 Quantitative Results

Table 1 presents the quantitative comparison of our proposed pipeline against the STTN[5] baseline. For this evaluation, our pipeline utilized traditional patch-based synthesis for spatial hallucination, operating strictly causally (relying only on past frame data via the motion-compensated memory buffer).

Table 1. Quantitative evaluation of background synthesis methods over a 179-frame sequence.

Synthesis Method	PSNR (dB)	SSIM	Processing Hardware
Baseline (STTN)	21.31	0.8086	Cloud (NVIDIA Tesla T4 GPU)
Proposed (Patch-Based)	20.51	0.7087	Apple M1 CPU

The baseline STTN[5] model achieves higher structural fidelity, with an SSIM of 0.8086 and a PSNR of 21.31 dB. This performance gap is expected due to fundamental differences in both algorithmic access to temporal data and computational resources. STTN[5] utilizes a non-causal Spatial-Temporal Transformer architecture that processes entire volumes of video frames simultaneously, leveraging future information to seamlessly fill in missing background regions. Furthermore, the baseline was accelerated using a cloud-based NVIDIA Tesla T4 GPU.

In contrast, our proposed pipeline achieves highly competitive metrics (20.51 dB PSNR) while operating under strict causal and edge-hardware constraints. It utilizes a traditional spatial search (PatchMatch[3]) to synthesize unknown regions on a local consumer-grade Apple M1 CPU. The ability to maintain acceptable structural fidelity without the benefit of future frame data, learned deep-learning priors, or discrete GPU acceleration underscores the efficiency of the proposed memory-buffer routing architecture.

4.4 Qualitative Analysis

A visual comparison of the synthesis results (Figure 3 and Figure 4) reveals distinct trade-offs

between the non-causal deep learning baseline and our proposed causal pipeline.

The STTN[5] baseline produces high temporal smoothness as shown in the middle column of Figure 4. However, because it relies on learned transformer priors to hallucinate missing regions, the resulting background often exhibits a painterly blur. Fine textures, such as the vertical folds in the curtain and the grain of the wooden door, lose their high-frequency details. This results in a visually soft reconstruction of the environment.

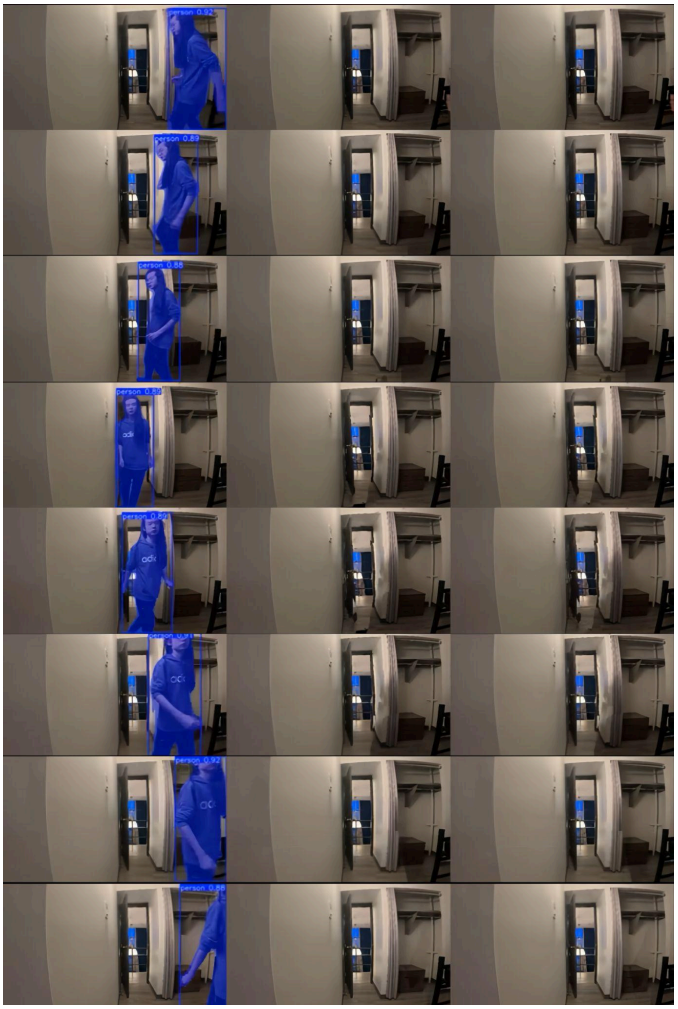


Figure 3: Stages of the proposed real-time inpainting pipeline. From left to right: (1) Input frame with YOLO-based semantic segmentation; (2) Temporal memory buffer containing motion-compensated historical background data; (3) Final output after causal patch-based synthesis. Original video was uploaded on github repository.

In contrast, the proposed patch-based pipeline shown in the right column of Figure 4 maintains significantly higher texture sharpness. By sampling directly from the motion-compensated memory buffer visualized in the center of Figure 3, the system preserves the original high-frequency details of the environment. While the PSNR is slightly lower due to minor staircase artifacts along high-contrast boundaries, such as where the subject silhouette meets the curtain, the perceptual clarity of the background is often superior to the baseline.

A notable limitation of the current pipeline is its dependency on the segmentation mask. The YOLO-based detector occasionally fails to mask fast-moving extremities. Because the architecture is strictly causal and does not use deep-learning-based hole filling priors, these unmasked regions are not sent to the synthesis module, resulting in localized ghosting artifacts. While these specific failures are difficult to capture in static frames, readers are encouraged to view the full video sequences on our GitHub repository to observe these edge cases in motion. Despite these limitations, the system demonstrates that a memory-buffer architecture can achieve competitive visual results on local CPU hardware without the need for future-frame lookahead.

5. Flow-Guided Video Inpainting Methods (E2FGVI)

In this section potential improvements to the E2FGVI[1] architecture are considered to improve reconstruction performance and we support these improvement suggestions with strong empirical results of an undertrained version of the original model and comparing it with the improved version of the model which is also undertrained (This is talked about in more detail in section 5.2.2). Before we go over the improvements and empirical results,

we have to give a more indepth explanation of how E2FGVI[1] works (Note we don't go over everything regarding E2FGVI, but we go over key details enough to understand the improvements we are trying to make and give a decent understanding of how E2FGVI works).

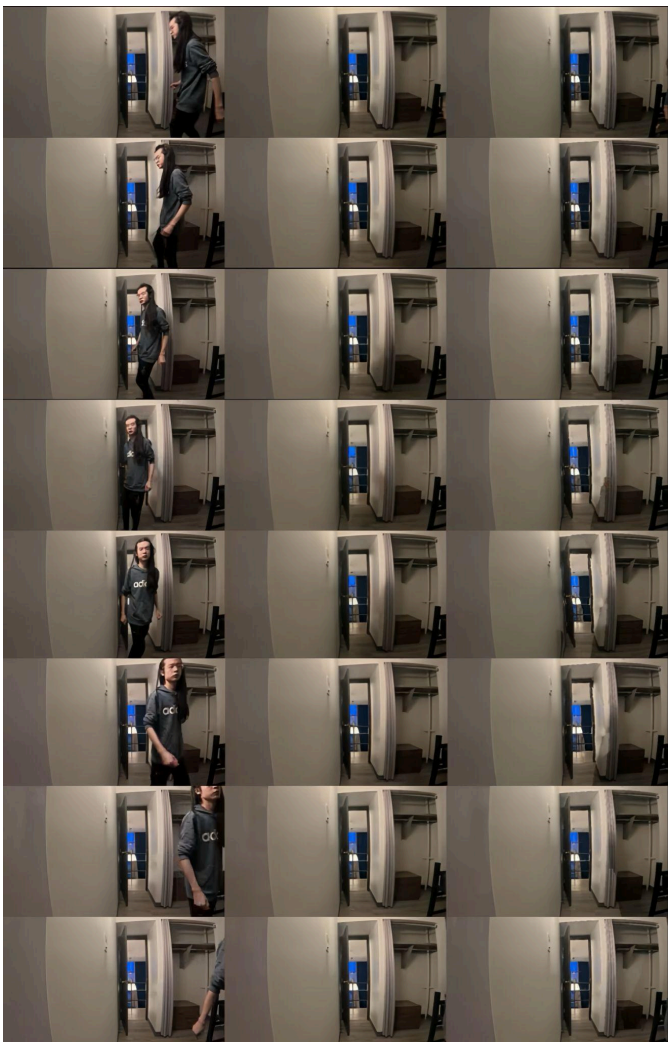


Figure 4. Sequential comparison of background synthesis results across 179 frames. From left to right: (1) Original input frame with foreground subject; (2) Results from the non-causal STTN[5] baseline; (3) Results from our proposed causal, patch-based pipeline. Original video was uploaded on github repository.

5.1 E2FGVI Summary

An overview of the E2FGVI [1] architecture is provided in Figure 2. To summarize, the model accepts an input video sequence and a corresponding set of binary masks, which define the regions requiring inpainting. Prior to inference, these masks are applied to the video frames to isolate the occluded areas. Additionally, a subset of non-local frames is sampled from the sequence to provide temporal context, which the model subsequently utilizes during the synthesis and hallucination stages.

5.1.1 Encoding To Feature Space

The first layer of E2FGVI[1] is the encoding layer, in this layer both the full corrupted video and the non-local corrupted frames are downsampled spatially by 1/4th as to make computations easier, but on top of this the video is transformed to something the paper refers to as "feature space". What feature space means is that each pixel in the video which usually is represented in 3 dimensions (RGB) are now represented in 128 dimensions this is done so each pixel can encode the specific features that are at this pixel as well encoding structural information local to that pixel (for example structural information about objects in the frame)

5.1.2 Optical Flow Calculation

The initial stage of the E2FGVI [1] architecture is the encoding layer. In this phase, both the occluded video sequence and the non-local frames undergo a $1/4\times$ spatial downsampling to reduce computational complexity. Furthermore, the input is projected into a high-dimensional feature space. In this context, feature space implies that the standard 3-channel RGB representation of each pixel is transformed into a 128-dimensional vector. This embedding enables each pixel to represent its inherent semantic features while simultaneously encoding structural information local to that pixel, such as object boundaries, textures, and spatial gradients within the frame.

5.1.3 Feature Propagation

Following the computation of optical flow, the resulting motion vectors are utilized to propagate features from the occluded local video sequence into the masked regions. Specifically, features from temporally adjacent previous and subsequent frames are aligned with the current frame using spatial warping guided by the previously calculated optical flow. Multiple cascading convolutional layers then process the concatenated warped frames along with the optical flow and the current frame. This stage is designed to learn optical flow offsets for error compensation as well as a weight mask to determine feature priority during the inpainting process.

A deformable convolutional layer is subsequently applied taking the weight mask, the flow offsets, and the adjacent frames as input. The output of this operation is integrated into the current frame and passed through a sequence of convolution and Leaky ReLU layers to yield the final representation with propagated features. This bidirectional process is repeated for each frame in the sequence to generate two distinct sets of propagated features per frame. Finally, these two feature maps are merged through a 1×1 convolution layer to obtain the unified output for the feature propagation stage.

5.1.4 Hallucination

Following the feature propagation stage, the remaining occluded regions are processed using a temporal focal transformer consisting of eight layers of multi-headed temporal focal self-attention. The transformer accepts two primary inputs: the local video frames generated during the feature propagation stage and the non-local video frames previously processed by the encoder. These frames are stacked and tokenized through a soft-splitting operation.

The resulting tokens are organized into $2 \times 2 \times 2$ groups known as subwindows, which serve as the fundamental units for temporal focal self-attention. During this process, multi-headed self-attention is applied to the tokens within each subwindow. To capture a broader context, the model attends to these subwindows alongside increasingly coarse video representations. These representations are generated by downsampling and aggregating neighboring subwindows, allowing the transformer to incorporate both local detail and global structure when hallucinating missing pixels.

5.1.5 Decoding

Upon completion of the hallucination stage, the occluded regions of the video are fully inpainted; however, the sequence remains at one-fourth of the original spatial resolution and resides within the high-dimensional feature space. To reconstruct the final output, this representation is passed through a decoder module. The decoder utilizes a sequence of deconvolution, Leaky ReLU, and 2D convolutional layers to perform spatial upsampling and map the 128-dimensional features back into the standard RGB color space. This decoding process yields the final inpainted video at the original input resolution.

5.2 Improvements

Now we make some potential improvements that can be made to improve the reconstruction metrics of E2FGVI[1].

5.2.1 Optical Flow Estimation

The effectiveness of feature propagation is fundamentally constrained by the quality of the motion estimation and remains dependent on the accuracy of the underlying optical flow field. Although the joint optimization of flow estimation and feature propagation helps mitigate certain errors, inaccuracies in motion estimation often persist and negatively impact the final reconstruction quality.

One significant improvement involves the integration of a more robust optical flow architecture. As demonstrated in the Empirical Results section, we substitute the original optical flow module used in E2FGVI [1] with a state of the art network known as RAFT [10]. This replacement allows for more precise feature alignment, particularly in scenarios involving fast motion or complex occlusions.

5.2.2 Decoding Improvements

Another critical architectural improvement implemented in this work is the addition of skip connections between the early layers of the encoder and the corresponding layers in the decoder. These connections facilitate the decoding process by providing high-resolution spatial information that is typically lost during the $1/4\times$ spatial downsampling operation. By bypassing the bottleneck of the transformer, these skip connections allow the decoder to better preserve fine structural details. This results in a more precise reconstruction when mapping the inpainted feature space back into the original video resolution.

E2FGVI Upgrades Code Available At ***https://github.com/zzprimal/E2FGVIUpgrades/tree/master***

5.3 Empirical Results

As established in the original E2FGVI [1] work, the baseline model was trained for 500,000 iterations using a batch size of 8 on 8 NVIDIA Tesla V100 GPUs. Due to temporal and computational constraints, replicating this scale of training for the improved architecture was not feasible. To ensure a fair comparative analysis, we established a localized baseline by training the original E2FGVI model for 25,000 iterations with a batch size of 8 on a single NVIDIA RTX 4090. We then compared these metrics against our improved version, which was trained for 100,000 iterations under identical batch settings. The resulting empirical data and performance improvements are detailed in Table 2.

	Accuracy			
	YouTube-VOS		DAVIS	
Models	PSNR	SSIM	PSNR	SSIM
E2FGVI	29.63	0.9446	29.09	0.9434
E2FGVI Improved	30.81	0.9541	30.10	0.9533

Table 2. Table showing the comparisons in metrics PSNR and SSIM between the E2FGVI model and E2FGVI model with our improvements

As demonstrated in Table 2, the proposed modifications achieved consistent improvements across all evaluated reconstruction metrics. While computational constraints precluded a comparison between fully converged versions of these two architectures, the current data provides strong empirical evidence that the integration of RAFT and encoder-decoder skip connections significantly enhances synthesis performance. These results validate the effectiveness of the architectural adjustments in improving overall reconstruction quality even within a reduced training schedule.

6. Discussion & Future Work

The proposed pipeline achieves real-time performance on edge devices, however, as discussed earlier, several limitations still remain. First, the system heavily depends on the quality of the segmentation mask. In particular, fast-moving regions such as limbs may not be fully captured, leading to artifacts such as ghosting or incomplete object removal.Second, due to the strictly real-time and causal design, the system cannot access future frames,so the reconstruction quality is inherently limited. In contrast, offline methods such as STTN[5] and E2FGVI[1] leverage global temporal

context across frames, enabling more consistent and accurate reconstruction.

These limitations suggest several directions for future work. One potential direction is to selectively incorporate neural networks into the pipeline, applying them only to unknown regions. This could improve reconstruction quality while maintaining low computational cost. Another direction is to improve optical flow estimation and motion compensation, which may reduce alignment errors and improve the stability of the memory buffer.

In conclusion, we believe the future of real-time video inpainting lies in hybrid approaches that combine the efficiency of classical methods with the generative capabilities of deep learning.These approaches have the potential to achieve high-quality reconstruction while satisfying real-time constraints.

References

- [1] Li, Zhen, Cheng-Ze Lu, Jianhua Qin, Chun-Le Guo, and Ming-Ming Cheng. "Towards an end-to-end framework for flow-guided video inpainting." In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 17562-17571. 2022.
- [2] Barath, Daniel, Jana Noskova, Maksym Ivashechkin, and Jiri Matas. "MAGSAC++: A fast, reliable and accurate robust estimator." In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 1304–1312, 2020.
- [3] Barnes, Connelly, Eli Shechtman, Adam Finkelstein, and Dan B. Goldman. "PatchMatch: A randomized correspondence algorithm for structural image editing." ACM Transactions on Graphics (ToG) 28, no. 3 (2009): 1–11.
- [4] R. Suvorov et al., “Resolution-robust Large Mask Inpainting with Fourier Convolutions,” arXiv: Computer Vision and Pattern Recognition, 2021
- [5]Zeng, Y., Fu, J., and Chao, H., “Learning Joint Spatial-Temporal Transformations for Video Inpainting,” in Proc. ECCV, 2020.
- [6] Gao, Chen, Ayush Saraf, Jia-Bin Huang, and Johannes Kopf. "Flow-edge guided video completion." In European Conference on Computer Vision, pp. 713-729. Cham: Springer International Publishing, 2020.
- [7] Xu, Rui, Xiaoxiao Li, Bolei Zhou, and Chen Change Loy. "Deep flow-guided video inpainting." In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 3723-3732. 2019.
- [8] E. Rublee, V. Rabaud, K. Konolige and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," 2011 International Conference on Computer Vision, Barcelona, Spain, pp. 2564-2571.2011
- [9] Fischler, Martin A., and Robert C. Bolles. "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography." Communications of the ACM, pp. 381 - 395. 1981.
- [10] Teed Z, Deng J. Raft: Recurrent all-pairs field transforms for optical flow. InEuropean conference on computer vision 2020 Aug 23 (pp. 402-419). Cham: Springer International Publishing.